

0.31. Deux lettres à la fois

Le 0.30, plus une deuxième lettre : deux `un_pas` dans la même boucle, et les deux bougent ensemble.

1. Ce qu'il faut comprendre

C'est le 0.30, plus une lettre, le B, qui descend pendant que le A avance.

Ce qui est nouveau ici : deux appels à `un_pas` dans la même boucle, l'un après l'autre. Comme `un_pas` n'attend pas, les deux se font **dans la même image** : les deux lettres bougent **ensemble**. Avec `deplace`, le B aurait attendu la fin du trajet du A.

Chaque lettre a ses deux variables : `xa`, `ya` pour le A ; `xb`, `yb` pour le B.

Le B s'arrête en ligne 16 : le `if (yb < 16)` ne le laisse plus avancer après, pour qu'il ne touche pas les nombres de la ligne 17.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1717 octets** de cartouche.

1. UNE VARIABLE — XA

```
uint8_t xa = 0;      // le A : sa colonne...
```

2. UNE VARIABLE — YA

```
uint8_t ya = 0;      // ...et sa ligne. Il file vers la DROITE.
```

3. UNE VARIABLE — XB

```
uint8_t xb = 0;      // le B : sa colonne...
```

4. UNE VARIABLE — YB

```
uint8_t yb = 2;      // ...et sa ligne. Il file vers le BAS.
```

5. UNE VARIABLE — IMAGES

```
uint8_t images = 0;  // le chronomètre, en images
```

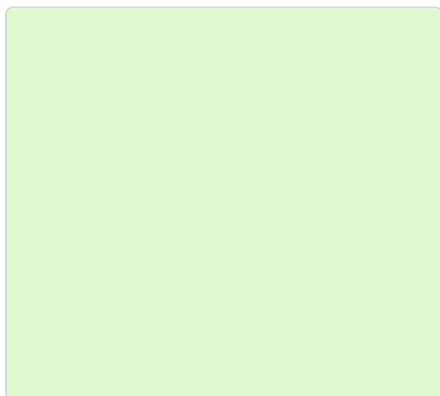
6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(xa, ya, ALPHABET[0]); // les deux lettres à leur place de départ
    poser(xb, yb, ALPHABET[1]); // ALPHABET[1] : la 2e lettre, B

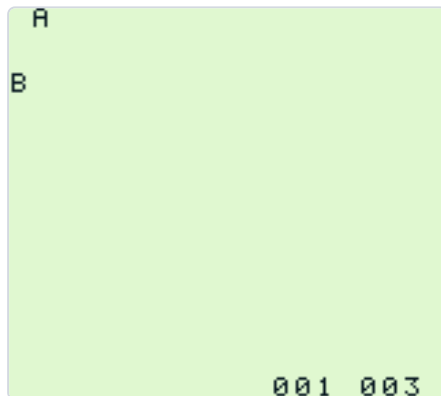
    while (true) {
        image();
        images++;
        if (images == 15) {           // 4 fois par seconde :
            images = 0;
            un_pas(xa, ya, ALPHABET[0], 1, 0); // le A : un pas à droite
            // La ligne ajoutée : le B, un pas en bas – DANS LA MÊME IMAGE que le A,
            // puisque un_pas n'attend pas. Tant que yb < 16 seulement.
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
            nombre(12, 17, xa); // la colonne du A, pour la voir changer
            nombre(16, 17, yb); // la ligne du B
        }
    }
}
```

3. Ce qui se passe, étape par étape

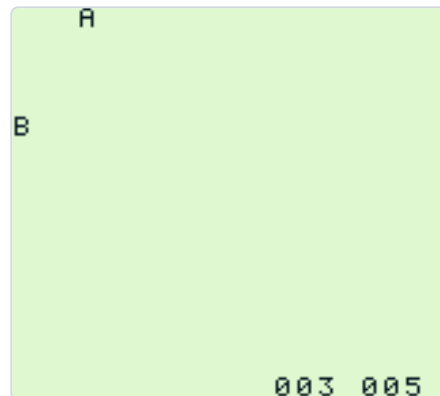
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



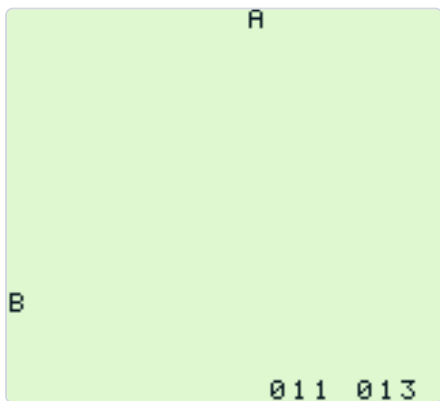
la toute première image — la console vient de commencer — image 1



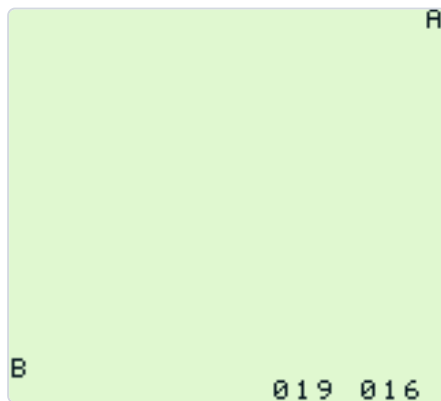
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui file vers la droite sur la ligne du haut, et un B qui descend en même temps sur la gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A et le B avancent (4 et 4)
 - ✓ du même nombre de pas
 - ✓ le A s'arrête au bord : colonne 19 ($x_a = 19$)
 - ✓ le B s'arrête en ligne 16 ($y_b = 16$)
-

Ensuite : **0.31.1. Deux lettres à la fois — de base, ailleurs** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++